

BANK LOAN ANALYTICS PROJECT

1. PROJECT OVERVIEW

This project analyzes bank loan performance using Power BI, SQL, Python, and Excel. The dashboard helps financial institutions monitor loan applications, funded amounts, repayment performance, loan quality, customer demographics, and lending trends. It enables stakeholders to identify high-risk loans, evaluate portfolio performance, and support data-driven lending decisions.

2. TOOLS USED

Tool	Purpose
Excel	Data Cleaning & Preparation
SQL Server	Data Querying & Analysis
Python	Data Cleaning, EDA & Visualization
Power BI	Interactive Dashboard Development
DAX	KPI Calculations & Measures

3. KEY KPIs

- Total Loan Applications: 38.6K
- Total Funded Amount: ₹435.8M
- Total Amount Received: ₹473.1M
- Average Interest Rate: 12.0%
- Average DTI: 13.3%
- Good Loan Percentage: 86.2%
- Bad Loan Percentage: 13.8%

4. DASHBOARD INSIGHTS

- . Summary Dashboard

- Total Loan Applications reached 38.6K.
- Total Funded Amount was ₹435.8M.
- Total Amount Received was ₹473.1M.
- Good Loans account for 86.2% of the portfolio.
- Bad Loans account for 13.8% of total applications.
- Fully Paid loans generated the highest repayment

- Overview Dashboard

- California received the highest funded amount.
- Debt Consolidation was the most common loan purpose.
- Mortgage customers received the highest funding.
- 36-month loans dominated the portfolio.
- Employees with 5 years of experience received the highest funded amount.
- Loan funding increased steadily throughout the year.

- Details Dashboard

Provides transaction-level analysis including:

- Loan ID
- Purpose
- Home Ownership
- Grade
- Sub Grade
- Issue Date
- Funded Amount
- Interest Rate
- Installment

5. SAMPLE SQL QUERIES

/* Total Loan Application */

select count(*) as total_loan_application from Bank_loan_data;

-- loan application from bank in last 12 months from current year --

select count(*) as MTD_total_LoanApp from Bank_loan_data

where month(issue_date) = 12 and year(issue_date) = 2021;

/* TOTAL LOAN AMOUNT */

SELECT SUM(loan_amount) as total_funded_amount from Bank_loan_data

where month(issue_date) = 12 and year(issue_date) = 2021;

SELECT SUM(loan_amount) as total_funded_amount from Bank_loan_data

where month(issue_date) = 11 and year(issue_date) = 2021;

/* total amount recieved from customer */

select sum(total_payment) as total_amount_received from Bank_loan_data;

SELECT SUM(total_payment) as MTD_total_payment from Bank_loan_data

where month(issue_date) = 12 and year(issue_date) = 2021;

SELECT SUM(total_payment) as PMTD_total_payment from Bank_loan_data

where month(issue_date) = 11 and year(issue_date) = 2021;

/* average intrest rate */

select Round(avg(int_rate),4) * 100 as avg_intrest_rate from Bank_loan_data

select Round(avg(int_rate),4) * 100 as MTD_avg_intrest_rate from Bank_loan_data

where month(issue_date) = 12 and year(issue_date) = 2021;

```
select Round(avg(int_rate),4) * 100 as PMTD_avg_intrest_rate from Bank_loan_data
where month(issue_date) = 11 and year(issue_date) = 2021;
```

```
ALTER TABLE Bank_loan_data
```

```
ALTER COLUMN dti DECIMAL(10,4);
```

```
select Round(avg(dti),4) * 100 as PMTD_avg_dti from Bank_loan_data
where month(issue_date) = 11 and year(issue_date) = 2021;
```

```
select Round(avg(dti),4) * 100 as MTD_avg_dti from Bank_loan_data
where month(issue_date) = 12 and year(issue_date) = 2021;
```

```
/* check good or bad loan from Bank_loan_data */
```

```
/* using case fuction to check the no of good / bad loans */
```

```
select
```

```
    (count(case when loan_status = 'Fully Paid' or loan_status = 'Current' then id
end)* 100)/
```

```
    count(id) as good_loan_percentage
```

```
from Bank_loan_data
```

```
/* how many good loan applications */
```

```
select count(id) from Bank_loan_data
```

```
where loan_status = 'Fully Paid' or loan_status = 'Current'
```

/* find the good loan funded amount */

```
select sum(loan_amount) as good_loan_fund_amount from Bank_loan_data
where loan_status = 'Fully Paid' or loan_status = 'Current'
```

/* find the payment amount recieved against the good loan */

```
select sum(total_payment) as good_loan_amount_recived from Bank_loan_data
where loan_status = 'Fully Paid' or loan_status = 'Current'
```

/* checking the bad loan values */

Select

```
        (count(case when loan_status = 'Charged off' then id end) * 100.00)/
        count(id) as bad_loan_percentage
from Bank_loan_data
```

/* total no of application for bad loan */

```
select count(id) as bad_loan_app from Bank_loan_data
where loan_status = 'Charged off'
```

/* find the bad loan funded amount */

```
select sum(loan_amount) as bad_loan_fund_amount from Bank_loan_data
where loan_status = 'Charged off'
```

/* find the payment amount recieved against bad loan */

```
select sum(total_payment) as bad_loan_amount_recieved from Bank_loan_data
where loan_status = 'Charged off'
```

/* monthly trend with respective loan application and payment amount status */

SELECT * FROM Bank_loan_data;

SELECT

MONTH(issue_date) as month_number,

DATENAME(MONTH, issue_date) as month_name,

COUNT(id) as Total_loan_applications,

sum(loan_amount) as total_funded_amount,

sum(total_payment) as total_received_amount

from Bank_loan_data

group by month(issue_date), DATENAME(MONTH, issue_date)

order by MONTH(issue_date)

/* loan application with respective to states of country */

select

address_state,

count(id) as Total_loan_applications,

sum(loan_amount) as total_funded_amount,

sum(total_payment) as total_received_amount

from Bank_loan_data

group by address_state

order by count(id) desc

```
/* loan application with respect to terms/tenure */  
select  
    term,  
    count(id) as Total_loan_applications,  
    sum(loan_amount) as total_funded_amount,  
    sum(total_payment) as total_received_amount  
FROM Bank_loan_data  
GROUP BY term  
order by term
```

```
/* with respect to length of year */  
select  
    emp_length,  
    count(id) as Total_loan_applications,  
    sum(loan_amount) as total_funded_amount,  
    sum(total_payment) as total_received_amount  
FROM Bank_loan_data  
GROUP BY emp_length  
order by emp_length
```

```
/* purpose for taking the loan */  
select  
    purpose,  
    count(id) as Total_loan_applications,  
    sum(loan_amount) as total_funded_amount,
```

```
        sum(total_payment) as total_received_amount
FROM Bank_loan_data
GROUP BY purpose
order by count(id) desc
/* with respective home ownership */
select
    home_ownership,
    count(id) as Total_loan_applications,
    sum(loan_amount) as total_funded_amount,
    sum(total_payment) as total_received_amount
FROM Bank_loan_data
GROUP BY home_ownership
order by count(id) desc
```

6. SAMPLE PYTHON ANALYSIS

I) importing liabrararies

```
import numpy as np
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
import warnings
```

```
import plotly.express as px
```

II) importing Datasets

```
Bank_df = pd.read_csv("C:/Users/charp/OneDrive/Desktop/Data Analytics  
projects/Bank Loan Monitoring Project/financial_loan.csv")
```



```
Print(Bank_df)
```

KPI Analysis

- Total Loan Application

```
Total_loan_application = Bank_df['id'].count()
```

```
print("Total Loan Application :", Total_loan_application)
```

- MTD Total loan Application

```
clean_df = Bank_df[Bank_df['issue_date'].notna()]
```

```
latest_issue_date = clean_df['issue_date'].max()
```

```
latest_month = latest_issue_date.month
```

```
latest_year = latest_issue_date.year
```

```
mtd_data = clean_df[(clean_df['issue_date'].dt.year == latest_year) &  
(clean_df['issue_date'].dt.month == latest_month)]
```

```
mtd_loan_application = mtd_data['id'].count()
```

```
print(f"MTD Loan Application(for {latest_issue_date.strftime('%B  
%Y')}):{mtd_loan_application}")
```

- Total Funded Amount

```
total_funded_amount = Bank_df['loan_amount'].sum()
```

```
total_funded_amount_million = total_funded_amount / 1000000
```

```
print("Total Funded Amount:{:.2f}M" .format(total_funded_amount_million))
```

- MTD Total Funded Amount

```
latest_issue_date = Bank_df['issue_date'].max()
```

```
latest_month = latest_issue_date.month
```

```

latest_year = latest_issue_date.year

mtd_data = Bank_df[(Bank_df['issue_date'].dt.year == latest_year) &
(Bank_df['issue_date'].dt.month == latest_month)]

mtd_total_funded_amount = mtd_data['loan_amount'].sum()

mtd_total_funded_amount_millions = mtd_total_funded_amount / 1000000

print(MTD Total Funded Amount:{:.2f}M" .format(mtd_total_funded_amount_millions))

Total Amount Recived

total_amount_received = Bank_df['total_payment'].sum()

total__amount_received_million = total_amount_received / 1000000

print("Total Amount Received :{:.2f}M" .format(total__amount_received_million))

```

- MTD Total Amount Received

```

latest_issue_date = Bank_df['issue_date'].max()

latest_month = latest_issue_date.month

latest_year = latest_issue_date.year

mtd_data = Bank_df[(Bank_df['issue_date'].dt.year == latest_year) &
(Bank_df['issue_date'].dt.month == latest_month)]

mtd_total_amount_received = mtd_data['total_payment'].sum()

mtd_total_amount_received = mtd_total_amount_received / 1000000

print("MTD Total Amount Received :{:.2f}M" .format(mtd_total_amount_received))

```

- Average intrest rate

```

avg_int_rate = Bank_df['int_rate'].mean()*100

print("Averag Intrest Rate:{:.2f}%".format(avg_int_rate))

```

- Average Debt to income Ratio(DTI)

```
Average_DTI = Bank_df['dti'].mean()*100  
print("Average DTI:{:.2f}%".format(Average_DTI))
```

- Good Loan Metrics

```
good_loans = Bank_df[Bank_df['loan_status'].isin(["Fully Paid","Current"])]  
total_loan_application = Bank_df['id'].count()  
good_loan_application = good_loans['id'].count()  
good_loan_funded_amount = good_loans['loan_amount'].sum()  
good_loan_received = good_loans['total_payment'].sum()  
good_loan_funded_amount_millions = good_loan_funded_amount / 1000000  
good_loan_received_millions = good_loan_received / 1000000  
good_loan_percentage = (good_loan_application / total_loan_application)* 100  
print("Good Loan Application :", good_loan_application)  
print("Good Loan Funded Amount {:.2f}M".  
.format(good_loan_funded_amount_millions))  
print("Good Loan Total Received Amount {:.2f}M".  
.format(good_loan_received_millions))  
print("Percentage of Good Loan Application : {:.2f}%".format(good_loan_percentage))
```

- Bad Loan Metrics

```
Bad_loans = Bank_df[Bank_df['loan_status'].isin(["Charged Off"])]  
total_loan_application = Bank_df['id'].count()  
Bad_loan_application = Bad_loans['id'].count()  
Bad_loan_funded_amount = Bad_loans['loan_amount'].sum()  
Bad_loan_received = Bad_loans['total_payment'].sum()
```

```

Bad_loan_funded_amount_millions = Bad_loan_funded_amount / 1000000
Bad_loan_received_millions = Bad_loan_received / 1000000
Bad_loan_percentage = (Bad_loan_application / total_loan_application)* 100
print("Bad Loan Application :", Bad_loan_application)
print("Bad Loan Funded Amount :{: .2f}M" .format(Bad_loan_funded_amount_millions))
print("Bad Loan Total Received Amount :{: .2f}M" .format(Bad_loan_received_millions))
print("Percentage of Bad Loan Application : {: .2f}%" .format(Bad_loan_percentage))

```

- Monthly Trend by issue Date For Total Funded Amount

```

monthly_funded = (
    Bank_df.sort_values('issue_date')
    .assign(month_name = lambda x: x['issue_date'].dt.strftime('%b %Y'))
    .groupby('month_name', sort = False)['loan_amount']
    .sum()
    .div(1_00_000)
    .reset_index(name = 'loan_amount_millions')
)

plt.figure(figsize = (10,6))

plt.fill_between(monthly_funded['month_name'],monthly_funded['loan_amount_millions']
,color = 'skyblue', alpha = 0.5)

plt.plot(monthly_funded['month_name'],monthly_funded['loan_amount_millions'],color
='blue', linewidth =2)

for i, row in monthly_funded.iterrows():
    plt.text(i, row['loan_amount_millions'] + 0.1, f"{row['loan_amount_millions']: .2f}",
            ha = 'center', va = 'bottom', fontsize = 9, rotation = 0, color ='black')

```

```

plt.title('Total Funded Amount by Month', fontsize = 14)
plt.xlabel('Month')
plt.ylabel('Funded Amount (millions)')
plt.xticks(ticks=range(len(monthly_funded)), labels = monthly_funded['month_name'],
rotation = 45)
plt.grid(True, linestyle= '--',alpha = 0.6)
plt.tight_layout()
plt.show()

```

- [Regional Analysis by State for Total Funded Amount](#)

```

state_funding =
Bank_df.groupby('address_state')['loan_amount'].sum().sort_values(ascending = True)

state_funding_thousands = state_funding / 1000

plt.figure(figsize=(8,14))

bars = plt.barh(state_funding_thousands.index, state_funding_thousands.values, color
= 'blue')

for bar in bars:

    width = bar.get_width()

    plt.text(width + 0.2, bar.get_y() + bar.get_height()/2,
            f'{width:,.0f}k', va = 'center', fontsize = 9)

plt.title('Total Funded Amount by State (Thousands)')
plt.xlabel('Funded Amount(INR)')
plt.ylabel('State')
plt.tight_layout()
plt.show()

```

- **Loan Term Analysis by Total Funded Amount**

```
from matplotlib.patches import Circle

term_funding_millions = (Bank_df.groupby('term')['loan_amount'].sum())/1000000

plt.figure(figsize=(6,6))

plt.pie(

    term_funding_millions,

    labels = term_funding_millions.index,

    autopct=lambda p: f"{p:.1f}%\n{p*term_funding_millions.sum()/100:.1f}M",

    startangle = 90,

    wedgeprops = {'width': 0.4}

)

centre_circle = Circle((0,0), 0.60, fc='white')

plt.gca().add_artist(centre_circle)

plt.title("Total Funded Amount by Term (Millions)")

plt.show()
```

- **Employee Length by Total Funded Amount**

```
emp_funding = Bank_df.groupby('emp_length')['loan_amount'].sum().sort_values()/1000

plt.figure(figsize = (10,6))

bars = plt.barh(emp_funding.index, emp_funding, color ='purple')

for bar in bars:

    width = bar.get_width()

    plt.text(width + 5, bar.get_y() + bar.get_height() /2,

        f"{width:,.0f}K", va = 'center', fontsize = 9)

plt.xlabel("Funded Amount (Thousand)")

plt.title("Total Funded Amount by Employment Length")
```

```
plt.grid(axis = 'x', linestyle = '--', alpha = 0.5)
```

```
plt.tight_layout()
```

```
plt.show()
```

- Employee purpose by Total funded amount

```
emp_funding =
```

```
Bank_df.groupby('purpose')['loan_amount'].sum().sort_values(ascending=False)/1000
```

```
plt.figure(figsize = (14,7))
```

```
bars = plt.bar(emp_funding.index, emp_funding.values, color = 'brown')
```

```
for bar in bars:
```

```
    height = bar.get_height()
```

```
    plt.text(bar.get_x() + bar.get_width() /2, height + emp_funding.max() * 0.01,
```

```
            f"{height:,.0f}K", ha = 'center', fontsize = 9)
```

```
plt.title("Total Funded Amount by Employee Purpose")
```

```
plt.xlabel("Purpose")
```

```
plt.ylabel("Funded Amount (Thousands)")
```

```
plt.xticks(rotation=45, ha='right')
```

```
plt.grid(axis = 'y', linestyle = '--', alpha = 0.5,)
```

```
plt.ylim(0, emp_funding.max() * 1.10)
```

```
plt.tight_layout()
```

```
plt.show()
```

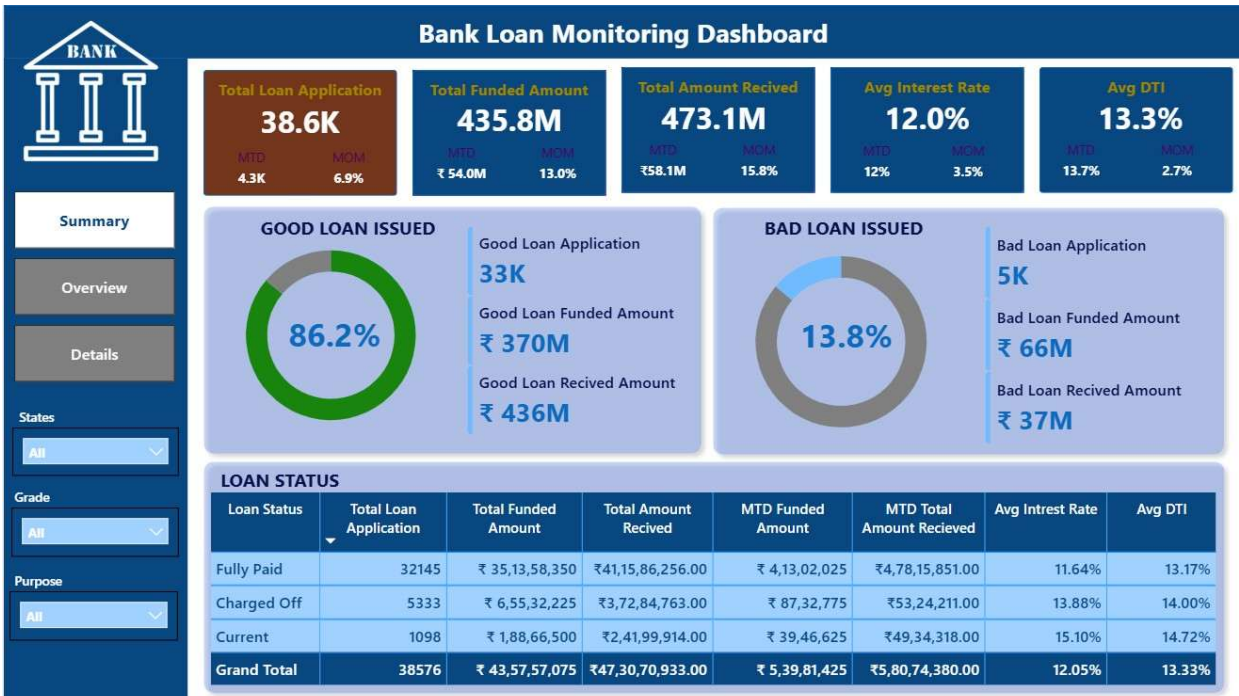
- Home ownership by Total Funded Amount

```
home_funding = Bank_df.groupby('home_ownership')['loan_amount'].sum().reset_index()
```

```
home_funding['loan_amount_millions'] = home_funding['loan_amount'] / 1000000
```

```
fig = px.treemap(  
    home_funding,  
    path = ['home_ownership'],  
    values = 'loan_amount_millions',  
    color = 'loan_amount_millions',  
    color_continuous_scale = 'Blues',  
    title = 'Total Funded Amount by Home Ownership (millions)'  
)  
fig.show()
```


7. DASHBOARD OVERVIEW





8. CONCLUSION

This project successfully transformed raw bank loan data into meaningful business insights using SQL, Python, Power BI, and DAX. The dashboard provides a comprehensive view of loan applications, funded amounts, repayments, customer segments, and loan quality. By monitoring KPIs such as Good Loan Percentage, Bad Loan Percentage, Interest Rate, and DTI, the solution helps financial institutions make informed lending decisions, reduce credit risk, and improve portfolio performance.